

Missouri River Bedform Analysis

Codebase Documentation

Data Importing, Multibeam and Single beam Processing,
and Data Visualization

*Developed for the U.S. Army Corps of Engineers – Kansas City District
Engineering Division, Hydrology Section*

Matthew Free

Last updated: 09/2025

Email: mfree1015@gmail.com

mrfqgk@umsystem.edu

TABLE OF CONTENTS

□ **Introduction**

□ **Importing Data**

2.1. File Structure

2.2. Loading Excel Files

2.3. Data Structure

□ **Processing Algorithms**

3.1. Loading

3.2. Processing

3.3. Saving

□ **Main Multibeam Workflow**

4.1. Outline

4.2. ArcGIS Script

4.3. Step-by-step Execution

□ **File Summary**

5.1. Drivers

5.2. Processing Functions

5.3. Plotting Functions

1. INTRODUCTION

The Missouri River Bedform Analysis Code was developed to provide robust, automated algorithms for the measurement and prediction of dune scales (height and length). These algorithms have been deployed in use for research of the Lower Missouri River (LMR), however are adaptable and widely expandable to any other River given the correct parameters. This codebase implements functionality for three main objectives: Importing Data, Processing Data, and Plotting Data. Ultimately, these three main objectives involve transforming raw bathymetric data supporting hydraulic modeling and river navigation.

The **Importing Data** section supports both single beam and multibeam datasets, each with distinct formatting requirements. Single beam data is imported from a single, structured Excel file, which limits flexibility but ensures consistency for datasets formatted in that specific layout. In contrast, the multibeam importing algorithm is designed to interface directly with output from the ArcGIS processing script. This structure relies on Excel files exported from ArcGIS and allows for streamlined importing.

The **Processing Algorithms** section applies many automated functions including: filtering, interpolation, smoothing, crest and trough detection, and ultimately dune characteristic extraction. Single beam and Multibeam Data are processed in the exact same manner, as multibeam data is broken into bed elevation profiles (BEPs) and processed as single beam.

The **Main Multibeam Workflow** demonstrates how to combine all sections as a single “pipeline.” This outlines all steps of the process beginning with scraping the data from eHydro, importing the data into MATLAB, processing the data, and then ending with plotting the data.

The **File Summary** section provides a summary of all code files that are included in this codebase. It details information on the driver scripts that chain together function calls in the proper order as well as the functions themselves, both for processing and plotting.

This document and codebase both collectively serve as a toolbox to process River data. This can be applied not only to historical surveys, as it had been for the LMR, but also to other Rivers and future data collection efforts. This document provides explanation of not only what custom functions have been created for this project but also details *where to find them and **how to use them.

*All paths to any files referenced are given. All paths are relative.

**All Function files contain a header with detailed function information.

2. IMPORTING DATA

2.1 File Structure

Single Beam Importing:

Excel file with three sheets named: "Left", "Center", "Right." Each of these contain columns of river mile, offset from sailing line, water surface elevation, and bed elevation for each date of collection. The order and location of these columns matter, for each field and each subsequent date of collection on all sheets, however the names at the top of each column do not. Selected columns are only naively read (based on known format) and then saved.

Path to Raw Data File: ".\FreeMoRiv\Single Beam Data\MoRiverData (Raw).xlsx"

Multibeam Importing:

Excel file with many different column headers. The ones necessary to run the importing function are: ORIG_FID, POINT_X, POINT_Y, POINT_Z. ORIG_FID gives the index of the profile, POINT_X gives the easting, POINT_Y, gives the northing, and POINT_Z gives the depth. These are standard excel sheet outputs from the ArcGIS scraping script (4.2). The order and location of these fields do not matter as long as they are present above a column of data.

Path to Example File Folder: ".\FreeMoRiv\Multibeam Data\MoRiver\ExcelSpreadsheets\"

2.2 Loading Excel Files

Single Beam Data Loading:

This MATLAB class structure file is run from a driver script found in the following file:

".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\SingleBeamDataLoading\dr_getData.m" This file groups all data into a class object and then reformats into a normal struct.

```
%% Load Data Object

% gets the data into a table from excel
% only need to call the constructor when using new data from excel
% this step may take around 5 minutes to read from excel
% .mat file used below contains the necessary rawData class structure
    %rawData = cMODataLoad();

% loads rawData.mat to retrieve object variable
    rawData = load("rawData.mat");
    rawData = rawData.rawData;
```

Figure 1. Shows a code snippet from the single beam loading file.

Figure 1 shows the lines of code that execute the class constructor which reads the data from excel. The constructor line has been commented out and below a load statement was added,

which loads the saved .mat file containing the class object. The output of this driver script is the “MoData” structure which indirectly interfaces with the bedform analysis and processing algorithms. The output data structure can be found saved in the following file:

“.\FreeMoRiv\Single Beam Data\MoRiverData.mat”

Multibeam Data Loading:

This MATLAB driver script can be found in the following file:

“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MultiBeamDataLoading\dr_getStackProfileData3D.m” This file first has two important paths, as shown in the code snippet in below in Figure 2.

```
addpath("..\MoRiver\ExcelSpreadsheets\");
addpath("..\MoRiver\OutputMATLABFiles\");
addpath("..\MoRiverBedformIdentification\");

%% Get File Names
% Excel File to Read
    outputPath = "...\MoRiver\OutputMATLABFiles\";
    inputPath = "...\MoRiver\ExcelSpreadsheets\";

    fileNames = getFilenames(inputPath);
```

Figure 2. Shows code snippet of path system.

The “addpath” function as well as the “outputPath” and “inputPath” variables correspond to one another and contain two different important paths. One relative path to the directory with excel spreadsheets output from the ArcGIS scraping script (4.2) as the input and one relative path to the desired destination directory for the output MATLAB files as output. The locations in Figure 2 have since been changed, and so the code has been updated for this reorganization of files.

Once run, this reads all the files within the excel spreadsheets directory and outputs “n*m” MATLAB files, where “n” is the number of indices from the ORIG_FID field within each file and “m” is the number of files run from input directory. Each output file is named with the following convention: “*basenameOfOriginalFile*”_“*ORIG_FID*”.mat This is explained in further detail below (2.3) and examples of these files can be found in the following directory:

“.\FreeMoRiv\Multibeam Data\MoRiver\OutputMATLABFiles\”

2.3 Data Structure

Single Beam Structure:

The output of the single beam driver script is the file “MoRiverData.mat” containing the MATLAB data structure variable “MoData.” This file can be found saved in the path referenced above (2.2) for Single Beam data loading.

MoData Structure is a 1x4 MATLAB Struct, each index corresponding to a date: 1, 2, 3, and 4 corresponding to April, May, June, and November accordingly. The structure within each index contains the following fields: Center, Left, and Right which designate where data was collected in the channel. Finally, within each of those fields there are the following subfields:

- x: vector of river mile location at each point (miles)
- z: vector of elevation values at each point (ft)
- offset: vector of offset from recommended sailing line at each point (ft)
- waterElevation: vector of water surface elevation values at each point (ft)
- date: date of collection in 2021
- size: number of elements in the data vector
- riverMileLength: length in miles of the reach (miles)

Name	Value	Size	Class
 MoData	1×4 struct	1×4	struct

 1×4 struct with 3 fields			
Fields	 Center	 Left	 Right
1	1×1 struct	1×1 struct	1×1 struct
2	1×1 struct	1×1 struct	1×1 struct
3	1×1 struct	1×1 struct	1×1 struct
4	1×1 struct	1×1 struct	1×1 struct

Field	Value	Size	Class
 z	154883×1 double	154883×1	double
 x	154884×1 double	154884×1	double
 offset	154884×1 double	154884×1	double
 waterElevatio	154884×1 double	154884×1	double
 date	"MAY 26"	1×1	string
 size	[154884,1]	1×2	double
 riverMileLengt	[45.6299;0]	2×1	double

Figure 3. Shows the MATLAB variable “MoData” in the workspace and the corresponding fields and the subfields below.

Multibeam Structure:

The output of the multibeam driver script is a batch of multiple files as described in the section above (2.2). Each file retains the base name of the file of origin. Each file of origin has multiple associated output files corresponding to the profile line taken. This is expressed as an index placed in the output file name following the base name used. An example of this is the input file “MO_01_LMR_20180327_CS_50_0_51_0_SORT_A.xlsx” which becomes “MO_01_LMR_20180327_CS_50_0_51_0_SORT_A_1.mat” for index one (sailing line). Each profile index staggers out from the sailing line until reaching profiles 20 and 21, which lie on the outer edges. Generally, all even profiles are on the northern side of the sailing line and odd profiles to the south.

Each individual profile MATLAB file contains only five variables:

- s: vector of distance along the profile (m)
- z: vector of pseudo-elevation values at each point (m)
- Easting_values: The original eastings (m)
- Northing_values: The original northings (m)
- Depth_values: The original depths at each point (ft)

Name	Value	Size	Class
 Depth_val...	96×1 double	96×1	double
 Easting_v...	96×1 double	96×1	double
 Northing...	96×1 double	96×1	double
 s	96×1 double	96×1	double
 z	96×1 double	96×1	double

Figure 4. Shows all MATLAB variables that are loaded into the workspace from each output file.

3. PROCESSING ALGORITHMS

3.1 Loading

Single Beam Algorithm Loading:

The single beam processing script can be found in the following file:

`“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoRiverData.m”`

. This demonstrates how first data is loaded by using MATLAB’s built in “load” function with the single beam data structure as described above (2.3). Then parameters are passed into the custom function “getRiverData”, which can simultaneously return data for two single beam lines from two months. If only one input month argument is given as shown in Figure 5, both sets of output vectors will identically contain that single month.

```
%% Variables
xStart = 132; % starting mile
xLength = 2; % plot length (miles)
month1 = "June";
Axis = "Center";

%% Parameters
primaryProminence = 0.75;
secondaryProminence = 0.01;

%% Get Data
load("MoRiverData.mat");

% get the x, y data
[x1, y1, x2, y2] = getRiverData(MoData, xStart, xLength, month1, Axis);
```

Figure 5. Loading data and providing input parameters to retrieve desired data.

Once the x and y vectors are obtained, processing steps such as smoothing, interpolation, and eventually bedform identification can be done.

Multibeam Algorithm Loading:

The multibeam processing script can be found in the following file:

`“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoRiverMultibeamData.m”`. This file shows how all files in a given directory are read. The most important first step is changing the “addpath” statement where all output MATLAB files were saved to, referenced above (2.2). In this case, they are saved in the folder “OutputMATLABFiles” as seen below in Figure 6. Next is to provide the “filenamePath” which would be the folder where all input excel files are located, as referenced above (2.2). Finally, a new directory should be given for the location of all output files as a save path.


```

addpath('..\MoRiverBedformIdentification\');
addpath('..\MultiBeamDataLoading\MoRiver\OutputMATLABFiles\');
addpath('..\MoRiverBedformIdentification\VectorMath\');
addpath('..\..\MAW MATLAB\');

%% Parameters
filenamePath = "..\MultiBeamDataLoading\MoRiver\ExcelSpreadsheets\";
savePath = "..\AlgorithmResultsMultibeam\MoRiver\ProminenceMeters\Prominence0.0900\";
fileNames = getFilenames(filenamePath);

%% Loop
for i = 1:length(fileNames)
    f = fileNames(i);
    f = f.split(".");
    filename = f(1);

    %% Get Data
    Data = getProfileData(filename, 75, 0.09*3.28084);

for i = 1:21
    file = sprintf("%s%s%s%s",filename, "_", num2str(i), ".mat");
    % Load file if it exists
    if exist(file, "file")
        load(file); %#ok<LOAD>

```

Figure 6. Code snippet of the driver script on top and the loop within the “getProfileData” function below.

This driver script loops through all the filenames found within the given folder and inputs only the base name into the “getProfileData” function. With this filename, the snippet below the driver in Figure 6 shows how this function uses the name to load and process each profile individually. With only the base filename given, the function checks if a profile exists and processes it if found.

3.2 Processing

Single Beam Processing:

Within the same driver script referenced above (3.1), processing can occur. First, data is smoothed lightly for better peak identification and noise reduction and then the x and y vectors are run through the peak identification function.

```

% get the x, y data
[x1, y1, x2, y2] = getRiverData(MoData, xStart, xLength, month1, Axis);

% data must be smoothed (sparingly) to distinguish peaks
[smoothX, smoothY] = curveSmoothing(x1, y1, 1, 25);

% Primary
propsPrim = duneHeightLength(smoothX, smoothY, primaryProminence);

% Secondary
propsSec = duneHeightLength(smoothX, smoothY, secondaryProminence);

```

Figure 7. Code snippet with processing steps for single beam data.

Figure 7 shows the processing steps, beginning with the function to retrieve data as referenced above (3.1). Once retrieved, the x and y vectors are run through a custom curve smoothing function and then through the bedform identification function with the prominence parameter in feet which outputs a data structure “propsPrim” containing all output data.

- peakIndices: indices of peaks on input vectors
- troughIndices: indices of trough on input vectors
- inclinedLength: straight line distance between troughs (m)
- verticalHeight: height from peak to intersection at inclinedLength (m)
- slope: from trough to trough (m/m)
- inclinedAngle: angle from trough to trough (degrees)
- lengthStoss: length from downstream trough to peak (m)
- lengthLee: length from peak to upstream trough (m)
- heightStoss: height from downstream trough to peak (m)
- heightLee: height from upstream trough to peak (m)
- totalLength: total sum of lee and stoss lengths (m)
- avgHeight: average of the lee and stoss heights (m)

Field	Value	Size	Class
peakIndices	1×147 double	1×147	double
troughIndices	1×146 double	1×146	double
inclinedLength	1×145 double	1×145	double
verticalHeight	1×145 double	1×145	double
slope	1×145 double	1×145	double
inclinedAngle	1×145 double	1×145	double
lengthStoss	1×145 double	1×145	double
lengthLee	1×145 double	1×145	double
heightStoss	1×145 double	1×145	double
heightLee	1×145 double	1×145	double
totalLength	1×145 double	1×145	double
avgHeight	1×145 double	1×145	double

Multibeam Processing:

Within the same driver script referenced above (3.1) processing also occurs simultaneously to the data loading function. In addition to the filename, all that is needed to run the function is a high-pass filter threshold and a prominence value.

```
Data = getProfileData(filename, 75, 0.09*3.28084);
```

Figure 8. Cope snippet showing example inputs for “getProfileData”.

The threshold parameter is a wavelet based high-pass filter which removes and frequency components exceeding the threshold given, in meters. The single beam peak finding algorithm is used on the multibeam profiles, meaning that the prominence parameter works exactly like

the single beam prominence. Prominence is a limiting parameter where any dunes measuring from trough to peak that are smaller are ignored. Prominence is input in feet. With these two parameters along with the filename, the “getProfileData” function outputs a data structure “Data” containing all output data.

- Eastings: vector of all Eastings
- Northings: vector of all Northings
- Depths: vector of all Depths
- Elevation: vector of all pseudo Elevations
- duneEastings: eastings at dune peaks and troughs
- duneNorthings: northings at dune peaks and troughs
- duneTroughDepths: depths at dune troughs
- dunePeakDepths: depths at dune peaks
- c1: vector of all dune heights
- c2: vector of all dune lengths
- propsPrim: array of propsPrim structures (as referenced above) for each profile
- primaryLength: vectors for the lengths in each profile
- primaryHeight: vectors for the heights in each profile
- pPeaks: vectors for the peak indices in each profile
- pTroughs: vectors for the trough indices in each profile
- avgLengths: vector of mean length in each profile.

Field	Value	Size	Class
 Eastings	<i>32777×1 double</i>	32777×1	double
 Northings	<i>32777×1 double</i>	32777×1	double
 Depths	<i>32777×1 double</i>	32777×1	double
 Elevation	<i>32777×1 double</i>	32777×1	double
▶  duneEastings	<i>1×1 struct</i>	1×1	struct
▶  duneNorthings	<i>1×1 struct</i>	1×1	struct
 duneTroughDepths	<i>2884×1 double</i>	2884×1	double
 dunePeakDepths	<i>2884×1 double</i>	2884×1	double
 c1	<i>2884×1 double</i>	2884×1	double
 c2	<i>2884×1 double</i>	2884×1	double
▶  propsPrim	<i>1×21 struct</i>	1×21	struct
 peakIndices	<i>[1×147 double, 1×147 d...</i>	-	-
 troughIndices	<i>[1×146 double, 1×147 d...</i>	-	-
 inclinedLength	<i>[1×145 double, 1×146 d...</i>	-	-
 primaryLength	<i>1×21 cell</i>	1×21	cell
 primaryHeight	<i>1×21 cell</i>	1×21	cell

3.3 Saving

Single Beam Data Saving

Within the same single beam driver script referenced above (3.1), saving is demonstrated with the “saveResultsToFile” function. In this function, various selected variables as well as some plots are saved in a preset directory.

```
% Get data from structs for histogram
primaryLength = propsPrim.totalLength; secondaryLength = propsSec.totalLength; primaryHeight = propsPrim
pPeaks = propsPrim.peakIndices; pTroughs = propsPrim.troughIndices; sPeaks = propsSec.peakIndices; sTroughs

%% Plot Data
plotDuneHeightLength(month1, smoothX, smoothY, pPeaks, pTroughs, primaryHeight, secondaryHeight, primaryLength)

%% Write Data
parameters = struct('primaryProminence', primaryProminence, 'secondaryProminence', secondaryProminence);
saveResultsToFile(xStart, xLength, month1, parameters, smoothX, smoothY, pPeaks, pTroughs, sPeaks, sTroughs,
```

Figure 9. Code snippet showing where data is saved in the driver script.

Multibeam Data Saving

Within the same multibeam driver script referenced above (3.1), saving is demonstrated with the “saveMultibeamResultsToFile” function. This function saves the data structure “Data” in the directory given by the save path variable.

```
%% Save Results
saveMultibeamResultsToFile(filename, Data, 1, savePath);
fprintf("Finished: %d/%d\n", i, length(fileNames))
close all;
```

Figure 10. Code snippet showing where data is saved in the multibeam driver script.

4. MAIN MULTIBEAM WORKFLOW

4.1 Outline

- Run the ArcGIS script
 - “.\FreeMoRiv\MATLAB\ArcGIS\Projects\ArcGISMultibeamScripting\ScrapelImportProcessExport.py”
- Run the csv to xlsx macro on the output files
 - “.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MultiBeamDataLoading\CSVtoXLSXmacro.xlsm”
- Run the excel files through the multibeam importing driver
 - “.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MultiBeamDataLoading\dr_getStackProfileData3D.m”
- Run the output MATLAB files through the multibeam processing driver
 - “.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoRivMultibeamData.m”
- Run the output data structures through the data exporting driver
 - “.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoExportData.m”

4.2 ArcGIS Script

The ArcGIS Script can be found in the following file:

“.\FreeMoRiv\MATLAB\ArcGIS\Projects\ArcGISMultibeamScripting\ScrapelImportProcessExport.py”, as well as in the ArcGIS project toolbox. Due to the nature of ArcGIS, running on a different computer does require some minimal setup of absolute paths.

```
# Get Coordinate System
crsInfo = getCRSFromZIP(z, r"C:\Users\g5edhmr-f\gdbCRS")

# Set up workspace
arcpy.env.workspace = r"C:\Users\g5edhmr-f\OneDrive - US Army Corps of Engineers\Documents\ArcGIS
```

Figure 11. Code snippet from scraping script for setup

The CRS save path first must be changed. It is located within the function “downloadAndExtractXYZAsCSV.” This is the one path that must not be located on an internet drive and must be located locally. Second, the workspace location needs to be changed. This should point to the project workspace location and this variable is located within the function “processAndExportCSV.” Once setup is complete, all that is left is defining the parameters.

At a high level, the ArcGIS scraping script only requires three parameters and four desired paths. All other logic is handled “under the hood” and is not necessarily important to its operation. Setup example can be found in Figure 11 below.

```

# Parameters
startDate = "2001-01-01 00:00:00"
boundingBox = (-93.6280018, 30.2334643, -89.4352473, 45.2359637)
lastRunURL = "https://ehydroprod.blob.core.usgovcloudapi.net/ehydro-surveys/CEMVP/UM_SP_P04_20190815_CS_7890_7900.ZIP"
urls = queryBoundingBox(startDate, boundingBox)
urls = sorted(set([feature["attributes"]["sourcedatalocation"] for feature in urls]))
#
# Variables
inputPathData = r"C:\Users\g5edhmr\OneDrive - US Army Corps of Engineers\Documents\ArcGIS\DataTables\MissRiver\Import"
inputPathMetadata = r"C:\Users\g5edhmr\OneDrive - US Army Corps of Engineers\Documents\ArcGIS\DataTables\MissRiver\Me
outputPath = r"C:\Users\g5edhmr\OneDrive - US Army Corps of Engineers\Documents\ArcGIS\DataTables\MissRiver\Export"
sailingLine = r"C:\Users\g5edhmr\OneDrive - US Army Corps of Engineers\Documents\ArcGIS\Projects\MultibeamScripting\M

```

Figure 12. Code snippet from the ArcGIS scraping script for inputs.

Shown in Figure 12, this script needs to be provided with a start date for the web search as well as a bounding box specified with latitude and longitude. This will provide bounds for a survey search on the eHydro website. A “lastRunURL” string variable is optional and is meant to be a way to pick back up from the last run survey when rerunning the script.

Also shown in Figure 12 are the necessary paths for running. “inputPathData” is the location where the raw downloaded data will be stored in csv files. “inputPathMetadata” is the location where all other information found within the downloaded file is saved to. “outputPath” gives the path to where all output csv files are saved to after processing. These are the files that will eventually be used in the MATLAB script. Finally is the path “sailingLine.” This path leads to an ArcGIS shape file of the recommended route line for the desired river. This modified shape file has been copied in the transverse direction to produce multiple lines parallel to one another along the river. These lines are then “densified” (which is a GIS function applied to the line before scripting) to interpolate additional points for the algorithm to sample from. The sample distance was set to 1m. Examples of these lines for the Missouri and the Mississippi can be found in the ArcGIS project file:

“.\FreeMoRiv\MATLAB\ArcGIS\Projects\ArcGISMultibeamScripting\ArcGISMultibeamScripting.aprx”.

4.3 Step-by-step Execution

1. ArcGIS script

The first step in the workflow is to run the ArcGIS scraping script to gather the raw profile data. Shown in Figure 12 above, this script will produce csv files in the path “outputPath.” These are the csv files that will be used in the next step.

Parameters:

- startDate: earliest date for web search (date-time)
- boundingbox: bounding box for web search (latitude/longitude)
- lastRunURL: script save state
- inputPathData: path for raw data
- inputPathMetadata: path for raw supplementary data
- outputPath: path for processed files
- sailingLine: modified - copy parallel (10m), densify (1m); path to this shape file

2. CSV to XLSX Macro

The next step in the workflow is to convert those csv files into xlsx files. MATLAB is capable of importing csv, however this transformation was necessary due to importing issues from format. Simply specify the paths in the VBA macro, and run. All excel files should be saved in a new directory which will be the input path for MATLAB functions.

Parameters:

- csvPath: path to the csv files
- savePath: path to the xlsx files

3. Multibeam Importing Driver

The third step in the workflow is to run the Import driver to produce all of the MATLAB files for each profile, as outlined above (2.2). All of these MATLAB files should be saved in a new directory to be used for the next step.

Parameters:

- outputPath: path for MATLAB files to be saved to
- inputPath: path where xlsx files are located

4. Multibeam Processing Driver

The fourth step in the workflow is to run the Processing Driver. The path that the MATLAB files from the above step were saved to should be used in this driver. Simply change the “addpath” function input to the location where these files are saved, as referenced above (3.1). Then specify the filename path which should be the save path from step 2 in this process. Finally, specify a new save path for the output data structures.

Parameters:

- filenamePath: path where xlsx files are located
- savePath: path where data structure files are to be saved to
- thresholdHPF: high pass filter threshold (m)
- prominence: dune height prominence value (ft)

5. Data Exporting Driver

This step is not necessary and the data from each individual survey can be used, however many of the algorithm result plots directly interface with this output structure and this is how the final data structure was formatted. To run this, change the very top variable in the file named “path.” This should be set to the save path from step 4. Second (optional), rename the data structure, which is currently set to “MoRiverData” as this was run on the Missouri River Data. Finally (optional), rename the export name, which is currently set to “MoRiverMultiBeamData.mat” at the bottom of the file.

Parameters:

- path: path to where data structures were saved after processing

5. FILE SUMMARY

5.1 Drivers

dr_loadUSGSData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\SingleBeamDataLoading\dr_loadUSGSData.m"

Purpose: To have text file gage data taken from USGS website saved in the project folder for use in the final exporting.

Survey Type: Single Beam/Multibeam

Operation Type: Temporary

dr_getData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\SingleBeamDataLoading\dr_getData.m"

Purpose: To take raw single beam data from a formatted spreadsheet and output a MATLAB data structure to be used in single beam processing.

Survey Type: Single Beam

Operation Type: Temporary

dr_getStackProfileData3D.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MultiBeamDataLoading\dr_getStackProfileData3D.m"

Purpose: To take raw profile data from ArcGIS script and output MATLAB data files to be used in multibeam processing.

Survey Type: Multibeam

Operation Type: Permanent

dr_FreeMoExportData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoExportData.m"

Purpose: To take output multibeam data structure files and combine into one cohesive data structure that interfaces directly with result plots and analysis.

Survey Type: Multibeam

Operation Type: Permanent

dr_FreeMoRiverData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoRiverData.m"

Purpose: Main single beam processing script. This script demonstrates importing, processing, plotting, and exporting.

Survey Type: Single Beam

Operation Type: Permanent

dr_FreeMoRiverDataParam.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoRiverDataParam.m"

Purpose: To test various input parameter values to optimize for the best value.

Survey Type: Single Beam

Operation Type: Temporary

dr_FreeMoRiverMultibeamData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoRiverMultibeamData.m"

Purpose: Main multibeam processing script. Used to import, process, plot, and export data for multibeam surveys.

Survey Type: Multibeam

Operation Type: Permanent

dr_FreeMoRiverMultibeamDataParam.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\dr_FreeMoRiverMultibeamDataParam.m"

Purpose: To test various input parameter values to optimize for the best value..

Survey Type: Multibeam

Operation Type: Temporary

dr_DiscardBadSurveys.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultibeam\dr_DiscardBadSurveys.m"

Purpose: Discards surveys under certain conditions and with listed names from QAQC.

Survey Type: Multibeam

Operation Type: Utility

dr_FreeBinnedAverages.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultibeam\dr_FreeBinnedAverages.m"

Purpose: Exports to excel from the data structure a summary of statistical data and average data using bins. Data bin size is set to 0.1 mi at default.

Survey Type: Multibeam

Operation Type: Utility

dr_FreeMultibeamPlots.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultibeam\dr_FreeMultibeamPlots.m"

Purpose: A driver script to run individual plot functions for the multibeam result data.

Survey Type: Multibeam

Operation Type: Utility

dr_FreeQAQC.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultibeam\dr_FreeQAQC.m"

Purpose: To create plots and export to file images of the plots to perform a QAQC analysis.

Survey Type: Multibeam

Operation Type: Temporary

dr_FreeProminenceSensitivity.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultibeam\Prominence Sensitivity\dr_FreeProminenceSensitivity.m”

Purpose: To create plots and export these images to file to compare different prominence values and determine qualitatively how sensitive they are.

Survey Type: Multibeam

Operation Type: Temporary

5.2 Processing Functions

areaAnalysis.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\BedFormAnalysis\areaAnalysis.m”

Purpose: Calculates the area between waveforms and finds the difference between them, both signed and unsigned

Survey Type: Single beam

Inputs:

- x1 = first x vector
- y1 = first y vector
- x2 = second x vector
- y2 = second y vector

Outputs:

- signed = signed difference in area
- unsigned = unsigned difference in area

averageElevation.m

Author: Single beam

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\BedFormAnalysis\averageElevation.m"

Purpose: Calculates the mean elevation for each month and prints the result.

Survey Type: Single beam

Inputs:

- month1 = first month of collection
- month2 = second month of collection
- y1 = y vector for the first month of collection
- y2 = y vector for the second month of collection

Outputs:

- N/A

offsetAnalysis.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\BedFormAnalysis\offsetAnalysis.m"

Purpose: Creates various plots to see how the offset from the sailing line may affect the data.

Survey Type: Single beam

Inputs:

- data = input "MoData" struct
- month1 = first month of collection
- month2 = second month of collection
- xStart = starting River Mile
- xLength = length of desired reach

Outputs:

- N/A

waveAnalysisDTW.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\BedFormAnalysis\waveAnalysisDTW.m"

Purpose: Takes in two line scans at different months and returns the total cost per unit length representing how similar the signals are.

Survey Type: Single beam

Inputs:

- y1 = first y vector
- y2 = second y vector

Outputs:

- D = total cost per unit length
- d = vector of cost path

calculateProminence.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\calculateProminence.m"

Purpose: Analyzes interpolated dune profiles using the wavelet transform to compute the average Fourier period and estimate dune prominence

Survey Type: Multibeam

Inputs:

- filename = filename for the desired survey
- thresholdHPF = high pass filter threshold value

Outputs:

- prominence = a recommended prominence value for the survey

computeRiverCurvature.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\computeRiverCurvature.m"

Purpose: estimates the signed curvature based on first and second derivatives

Survey Type: Multibeam

Inputs:

- eastings = vector of eastings
- northings = vector of northings

Outputs:

- curvature = vector of curvature values at each point

createMask.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\createMask.m"

Purpose: Is used to create a mask so that points can be removed after interpolation of a vector where there are large irregular gaps in data

Survey Type: Multibeam

Inputs:

- s = original s vector
- sInterp = interpolated s vector

Outputs:

- mask = logical masking vector

duneHeightLength.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\duneHeightLength.m"

Purpose: Processes profile or line scan data and returns dune information

Survey Type: Multibeam/Single beam

Inputs:

- X = input x vector
- Y = input y vector
- prominence = prominence value

Outputs:

- bedform = output structure of bedform information

duneStossAndLee.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\duneStossAndLee.m"

Purpose: Calculates the horizontal length and vertical height of the stoss and lee formed from the leading trough and peak as well as the peak and lagging trough.

Survey Type: Multibeam/Single beam

Inputs:

- x1 = x locations of troughs
- y1 = y locations of troughs

- x2 = x locations of peaks
- y2 = y locations of peaks

Outputs:

- lengthStoss = vector of stoss side lengths
- lengthLee = vector of lee side lengths
- heightStoss = vector of stoss side heights
- heightLee = vector of lee side heights

extractRiverMilesFromFilename.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\extractRiverMilesFromFilename.m”

Purpose: Uses a capture group to find and parse River Mile information within a filename.

Survey Type: Multibeam

Inputs:

- filename = input filename to search in

Outputs:

- riverMiles = vector of the start and end river miles found

getFileNames.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\getFileNames.m”

Purpose: To list all of the filenames within a directory

Survey Type: Multibeam

Inputs:

- inputPath = path to the desired directory

Outputs:

- fileNames = vector of the filenames within that directory

getGageLoc.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\getGageLoc.m"

Purpose: Converts multiple versions of the string data locations into the correct and detailed gage information

Survey Type: Multibeam

Inputs:

- gageLocIn = input gage location string variable

Outputs:

- gageLocOut = output gage location string variable information

getMonth.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\getMonth.m"

Purpose: Converts the string valued month into the correct index into the single beam data structure "MoData"

Survey Type: Single beam

Inputs:

- firstMonth = first month of collection
- secondMonth = second month of collection

Outputs:

- date1 = index for the first month of collection
- date2 = index for the second month of collection

getProfileData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\getProfileData.m"

Purpose: Gathers and processes multibeam data from a selected file

Survey Type: Multibeam

Inputs:

- filename = name of desired file to run
- thresholdHPF = high pass filter threshold
- primaryProminence = prominence value

Outputs:

- Data = output data structure containing dune information

getRiverData.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\getRiverData.m”

Purpose: Allows for easy access of data from the single beam data structure specifying various parameters such as the months, start, length, and frequency of the data

Survey Type: Single beam

Inputs:

- data = input data structure “MoData”
- xStart = starting River Mile
- xLength = desired length of reach
- firstMonth = first month of collection
- axis = location of collection “Left”, “Center”, or “Right”
- secondMonth = second month of collection
- frequency = frequency to interpolate on
- type = type of data “x” for elevation “o” for offset
- ox = padding amount to either side of data

Outputs:

- x1 = first x vector
- y1 = first y vector
- x2 = second x vector
- y2 = second y vector

getRiverMileFromCoordInterpolated.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\getRiverMileFromCoordInterpolated.m”

Purpose: takes an Easting and Northing coordinate (NAD83/UTM Zone 15) and interpolates a saved database to find the closest River Mile value

Survey Type: Multibeam

Inputs:

- Easting = single value or vector of input eastings
- Northing = single value or vector of input northings
- resolution = resolution interpolation of the database

- searchRadius = radius of search within the database to interpolate on

Outputs:

- riverMile = single value or vector of output river mile locations

maskData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\maskData.m"

Purpose: Uses a mask from the interpolated longitudinal vector and removes data points that lie within any part of the logical mask that is true

Survey Type: Multibeam

Inputs:

- props = input props data structure
- mask = logical masking vector

Outputs:

- props = output props data structure

saveMultibeamResultsToFile.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\saveMultibeamResultsToFile.m"

Purpose: To save and export data and plots for the multibeam algorithms

Survey Type: Multibeam

Inputs:

- filename = filename for saving
- Data = data structure to be saved
- s = save variable, "1" for yes and "0" for no
- path = path to save location

Outputs:

- N/A

saveResultsToFile.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\saveResultsToFile.m"

Purpose: To save and export data and plots for the single beam algorithms

Survey Type: Single beam

Inputs:

- xStart = starting River Mile location
- xLength = length of desired reach
- month1 = month of collection
- parameters = saved values for prominence
- smoothX = vector of interpolated X values
- smoothY = vector of interpolated Y values
- pPeaks = index locations of primary peak
- pTroughs = index locations of primary trough
- sPeaks = index locations of secondary peaks
- sTroughs = index locations of secondary troughs
- primaryHeight = vector of primary heights
- secondaryHeight = vector of secondary heights
- primaryLength = vector of primary lengths
- secondaryLength = vector of secondary lengths
- algorithm = algorithm used "Free"
- Axis = location of collection "Left", "Center", "Right"

Outputs:

- N/A

curveSmoothing.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\VectorMath\curveSmoothing.m"

Purpose: Uses cubic interpolation to smooth noisy data, returning smoothed vectors from the input data.

Survey Type: Multibeam/Single beam

Inputs:

- x = input x vector
- y = input y vector
- n = downsampling parameter. Determines the frequency of the new vector to interpolate to
- m = upsampling parameter. Determines the frequency of the output vector to interpolate to

Outputs:

- X = output smoothed x vector
- Y = output smoothed y vector

derivative.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\VectorMath\derivative.m”

Purpose: Differentiates and returns the slopes between each point

Survey Type: Single beam

Inputs:

- x = input x vector
- y = input y vector

Outputs:

- x = output x vector
- yo = output vector of slopes

integral.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\VectorMath\integral.m”

Purpose: Uses trapezoidal method to integrate and return the total sum and a vector of sums

Survey Type: Single beam

Inputs:

- x = input x vector
- y = input y vector

Outputs:

- s = total area under the curve
- area = vector of each trapezoidal area

map.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\VectorMath\map.m”

Purpose: Interpolates input points and maps to a desired vector

Survey Type: Multibeam/Single beam

Inputs:

- x = input x vector
- y = input y vector
- frequency = frequency of new vector
- xStart = starting value of new vector
- xLength = length value for the new vector

Outputs:

- X = output interpolated x vector
- Y = output interpolated y vector

pointSlopeHeight.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\VectorMath\pointSlopeHeight.m"

Purpose: Calculates the vertical distance between a dune peak and the intersection with the line formed by an inclined slope between the surrounding two troughs. Uses the point slope form equation for calculations.

Survey Type: Multibeam/Single beam

Inputs:

- x1 = x locations of troughs
- y1 = y locations of troughs
- x2 = x locations of peaks
- y2 = y locations of peaks

Outputs:

- height = vector of height values
- slope = vector of slope values

prominenceTest.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\VectorMath\prominenceTest.m"

Purpose: Allows for quick and easy testing of the prominence parameter. This function returns the number of dunes, average lengths and heights of dunes associated with a specified value of prominence.

Survey Type: Single beam

Inputs:

- X = input x vector
- Y = input y vector
- prominence = prominence value

Outputs:

- number = number of dunes located
- lengths = vector of dune lengths
- heights = vector of dune heights

pythagoreanTheorem.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\VectorMath\pythagoreanTheorem.m"

Purpose: Calculates the inclined length or the straight-line distance between each element in an x and y vector.

Survey Type: Multibeam/Single beam

Inputs:

- x = x vector
- y = y vector

Outputs:

- c = vector of straight-lined lengths between points

importData.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\SingleBeamDataLoading\importData.m"

Purpose: Used to read from a spreadsheet to fill a data structure

Survey Type: Single beam

Inputs:

- filename = excel file to read from
- sheet = sheet within that file to read from

Outputs:

- MOData_Struct = output data structure

- index = last index run

readExcelColumn.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\SingleBeamDataLoading\readExcelColumn.m”

Purpose: To read a single specified column from an excel sheet

Survey Type: Single beam

Inputs:

- filename = excel file to read from
- sheet = sheet within that file to read from
- column = column on that sheet to read
- cEnd = last row to read

Outputs:

- columnData = output data from read column

binData.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultiBeam\Plotting Functions\binData.m”

Purpose: Loads MO River multibeam data and bins the values for length, height, and flow depth using a specified bin size. It also computes means and percentiles (5, 25, 50, 75, 95) within each bin for further analysis

Survey Type: Multibeam

Inputs:

- binSize = size of each bin for the data
- dataStructure = input data structure

Outputs:

- AllRMS = vector of all binned river miles
- AllLs = vector of all binned lengths
- AllHs = vector of all binned heights
- Allhs = vector of all binned flow depths
- AllDates = vector of all binned dates
- AllFileNames = vector of all binned filenames
- AllQ = vector of all binned flows
- AllStats = vector of all binned stat data

curveFit.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti
beam\Plotting Functions\curveFit.m”

Purpose: Fits given data to power law and returns statistical information

Survey Type: Multibeam

Inputs:

- x = input x data
- y = input y data
- fitX = sample x vector for output fit curve

Outputs:

- a = power law coefficient
- b = power law power
- fitX = fit x vector
- fitY = fit y vector
- paramFit = 90 percent interval log space intercepts and stat information such as p, r^2 , and RMSE to show fit quality

getWtHPF.m

Author: Micah Wyssmann

Location:“.\FreeMoRiv\MATLAB\MAW MATLAB\getWtHPF.m”

Purpose: Performs a Wavelet based high pass filter to remove any frequency components greater than the threshold specified.

Survey Type: Multibeam

Inputs:

- z = original interpolated z vector
- ds = spacing between each s vector
- Lthresh = length threshold for the high pass filter

Outputs:

- zrec = reconstructed z vector after wavelet filter

getSprofileInterp.m

Author: Micah Wyssmann

Location:“.\FreeMoRiv\MATLAB\MAW MATLAB\getSprofileInterp.m”

Purpose: Interpolates to a standard frequency all Multibeam input data.

Survey Type: Multibeam

Inputs:

- sin = input s vector of longitudinal location along profile
- zin = input z vector of height at each s point
- ein = input easting vector
- nin = input northing vector
- din = input depth vector

Outputs:

- sout = output interpolated s vector
- zout = output interpolated z vector
- eout = output interpolated easting vector
- nout = output interpolated northing vector
- dout = output interpolated depth vector

5.3 Plotting Functions

elevationMap.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\elevationMap.m”

Purpose: Plots an elevation scatter map for Eastings and Northings. Allows for visualization of the riverbed to compare with the heatmap of dune scales.

Survey Type: Multibeam

Inputs:

- Eastings = vector of eastings
- Northings = vector of northings
- Elevation = vector of elevation values
- markerSize = size of the plot markers

heatMap.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\heatMap.m”

Purpose: Plots dune height and length in respective coordinate locations. Allows for visualization of dune scales from multibeam bathymetric data.

Survey Type: Multibeam

Inputs:

- Eastings = vector of eastings
- Northings = vector of northings
- c1 = vector of dune heights
- c2 = vector of dune lengths
- gridspacing = number of grid cells to bin the figure data

plotDataCompare.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\plotDataCompare.m"

Purpose: Plots two separate months over each other to compare

Survey Type: Single beam

Inputs:

- x1 = first x vector
- y1 = first y vector
- x2 = second x vector
- y2 = second y vector
- units = units of the data "imperial" or "metric"
- ratio = aspect ratio value

plotDuneHeightLength.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\plotDuneHeightLength.m"

Purpose: Creates a histogram for the primary heights and lengths as well as the secondary heights and lengths. Creates a plot of the bedform with the primary peaks plotted as red dots and the primary troughs plotted as blue dots. Prints the average primary and secondary dune heights and lengths.

Survey Type: Single beam

Inputs:

- month1 = month of collection
- X = x vector
- Y = y vector
- peakIndices = indices of dune peaks
- troughIndices = indices of dune troughs
- primaryHeight = vector of primary heights
- secondaryHeight = vector of secondary heights

- primaryLength = vector of primary lengths
- secondaryLength = vector of secondary lengths

plotLengthsVsTransverse.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\FreeAlgorithm\MoRiverBedformIdentification\plotLengthsVsTransverse.m"

Purpose: Plots average dune length in corresponding transverse location from sailing line.

Survey Type: Multibeam

Inputs:

- avgLengths = vector of the average length scale for each profile

boxAndWhiskersPlot.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultibeam\Plotting Functions\boxAndWhiskersPlot.m"

Purpose: Is used to visualize the quartiles for the variable y placed in the respective x longitudinal location

Survey Type: Multibeam

Inputs:

- variable = variable to be analyzed and plotted
- linewidth = thickness of plotted lines
- markerSize = size of the plot markers

plotBEPStack.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMultibeam\Plotting Functions\plotBEPStack.m"

Purpose: loads multibeam survey data, processes bathymetry, and generates multiple figures showing dune peaks and profiles.

Survey Type: Multibeam

Inputs:

- N/A

plotBradleyAndVenditti.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti beam\Plotting Functions\plotBradleyAndVenditti.m"

Purpose: Takes Length and Height values and plots against flow depth Allows for a scatter visualization of how length and height compare to flow. A fitted line is plotted over. Plots in relation to known and previously studied relationships.

Survey Type: Multibeam

Inputs:

- bin = dictates binning of data "off" or "on"
- scale = scale of the plot "lin" or "log"
- markerSize = size of the plot markers

plotDateVsRiverMile.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti beam\Plotting Functions\plotDateVsRiverMile.m"

Purpose: Plots a tiled layout of three plots containing a hydrograph of the discharge in (cfs) as well as date vs riverMile colored by the scale of dune height and length at that point.

Survey Type: Multibeam

Inputs:

- byLoc = create individual plots by location or not "on" or "off"
- markerSize = size of the plot markers

plotDuneScaleVsFlow.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti beam\Plotting Functions\plotDuneScaleVsFlow.m"

Purpose: Allows for a scatter visualization of how length and height compare to flow. A fitted line is plotted over.

Survey Type: Multibeam

Inputs:

- markerSize = size of the plot markers

plotDuneScaleVsGageHeight.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti
beam\Plotting Functions\plotDuneScaleVsGageHeight.m”

Purpose: Allows for a scatter visualization of how length and height compare to gage height. A least squares line is plotted over to show the trend.

Survey Type: Multibeam

Inputs:

- color = color of the data markers
- markerSize = size of the plot markers

plotGammaPDF.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti
beam\Plotting Functions\plotGammaPDF.m”

Purpose: Allows for visualization of alpha and theta for each survey compared with the gage height for each survey

Survey Type: Multibeam

Inputs:

- bin = dictates binning of data “off” or “on”
- color = color of the data markers
- markerSize = size of the plot markers

plotProminenceComparison.m

Author: Matthew Free

Location:“.\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti
beam\Plotting Functions\plotProminenceComparison.m”

Purpose: Loads three datasets of extracted dunes using different prominence thresholds (low, center, high). Calculates mean dune length and height for each dataset and compares the variability by computing the percent half range. The results are shown in histograms.

Survey Type: Multibeam

Inputs:

- N/A

plotScheiberAndFlemming.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti beam\Plotting Functions\plotScheiberAndFlemming.m"

Purpose: Allows for a scatter visualization of how length and height compare. A fitted line is plotted over. Plots in relation to known and previously studied relationships.

Survey Type: Multibeam

Inputs:

- bin = dictates binning of data "off" or "on"
- scale = scale of the plot "lin" or "log"
- markerSize = size of the plot markers

plotSteepnessVsFlowDepth.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti beam\Plotting Functions\plotSteepnessVsFlowDepth.m"

Purpose: Creates a log-log scatter plot of dune steepness (H/L) versus mean flow depth (h) using multibeam and single beam datasets. A power-law fit is applied to the multibeam data, and the curve is plotted with 90% confidence bounds. Single beam points are plotted for visual reference.

Survey Type: Multibeam

Inputs:

- N/A

qacHeatMap.m

Author: Matthew Free

Location: ".\FreeMoRiv\MATLAB\RiverBedformAnalysisAlgorithms\AlgorithmResultsMulti beam\Plotting Functions\qacHeatMap.m"

Purpose: Plots dune height and length in respective coordinate locations. Allows for visualization of dune scales from multibeam bathymetric data.

Survey Type: Multibeam

Inputs:

- Eastings = vector of eastings
- Northings = vector of northings
- Color = heatmap color vector, either dune heights or lengths

- gridspacing = number of grid cells to bin the figure data
- type = type of data "length" or "height"
- xBoundary = vector of x boundary line values
- yBoundary = vector of y boundary line values